OAUTH 2.0

API Testing for Beginners

Postman + Rest Assured | UserApisWithOauth2

Complete notes with code explanations, realistic scenarios, exercises, troubleshooting, and best practices

Learning goal

Understand why an access token is needed, how the local demo obtains and refreshes tokens, how to call protected Employee APIs, and how to automate the same flow using Rest Assured + TestNG.

Source materials used: OAuth2 Collection.postman_collection.json, OAuth2 Flow To Generate Access Token.xlsx, and Steps.txt. The examples below keep the project-specific URLs and response field names from those files, while clearly marking best-practice recommendations that go beyond the source.

Important: these are instructional notes. The local services were not started or executed while preparing this workbook. Confirm actual status codes and response fields against your running application before converting examples into pass/fail tests.

How to use this workbook

Read the sections in order and run the same request in Postman before automating it in Java. The best way to learn OAuth is to watch one full request travel from the token service to the protected service.

- 1. Project architecture and OAuth 2.0 vocabulary
- 2. Local setup and source-file differences to resolve
- 3. Postman collection: permissions, roles, users, and clients
- 4. Generate an access token and refresh it
- 5. Call protected Employee APIs and troubleshoot 401/403
- 6. Rest Assured + TestNG setup and reusable token support
- 7. End-to-end code examples and negative scenarios
- 8. Best practices, exercises, answers, and glossary

What you should know before starting

- Basic HTTP methods: GET reads data, POST creates or triggers an operation, PUT replaces/updates a resource, DELETE removes a resource.
- Basic JSON: key/value pairs such as {"name":"Sneha","salary":139999}.
- Basic Java: variables, methods, classes, maps, exceptions, and Maven dependencies are helpful. Every code concept is explained in plain language.
- Postman is used to explore requests manually. Rest Assured is used to automate those requests in Java and assert the results.

1. Understand the project architecture first

Your source material describes a local OAuth demo with two services running on different ports. They play different jobs, so do not confuse their base URLs.

Component	Base URL / endpoint	Responsibility
Authorization / user API service	http://localhost:9080	Manages users, roles, permissions, OAuth clients, token issuance, and token refresh in this tutorial.
Resource server	http://localhost:9081/employees	Hosts Employee APIs that require a valid access token.
Postman collection	OAuth2 Collection	Lets you issue setup, token, and protected-resource requests manually.
Rest Assured tests	Your Maven/TestNG project	Automates HTTP requests and checks expected response behavior.

The complete flow in simple words

→ Prepare the demo data: a user, role/permission configuration, and an OAuth client must exist as required by the local application.

→ Send a POST request to http://localhost:9080/oauth2/token using the client details and demo username/password. The local example expects accessToken and refreshToken in the response.

→ Save the access token in a Postman variable or Java variable. Do not paste a short-lived token into every request manually.

→ Call http://localhost:9081/employees with the header Authorization: Bearer <access-token>. Without a token, the supplied steps say the API returns 401 Unauthorized; with an accepted token, the example expects successful access.

→ When the access token expires, send the saved refresh token to http://localhost:9080/oauth2/refresh, then store the newly returned token values and retry the protected API.

Mental model

The token endpoint issues a credential; the employee endpoint checks that credential before returning or changing employee data. A token is not the employee data itself—it is proof of the access granted by the authorization system.

Authentication versus authorization

Term	Beginner explanation	Project example
Authentication	Who are you?	The token request checks the demo user and client credentials.
Authorization	What are you allowed to do?	The Employee API checks token validity and whether the caller can perform the requested operation.
Access token	Short-lived credential sent on API calls.	Stored as accessToken by the provided Postman script.
Refresh token	Credential used to request a new access token without repeating the same login step.	Sent in JSON to /oauth2/refresh in this project.
Scope	Named access permission requested/granted to a token.	The example requests read write.
Client ID / secret	Identifies and authenticates the API client to the token service.	employee_client / demo secret in the sample client setup.

401 versus 403

- 401 Unauthorized usually means credentials are missing, invalid, expired, malformed, or not accepted by the server. In everyday terms: the server cannot accept the authentication for this request.
- 403 Forbidden usually means the server recognized the caller but the requested action is not allowed. For example, a valid token may have read access but not write access. Confirm your local app follows this convention; its actual contract is authoritative.

A successful token response does not guarantee every API request will succeed. The token might be expired, meant for a different resource server, or lack the permission required for a particular endpoint.

2. Source-specific details and items to verify

The three reference files mostly describe the same training system, but a few examples differ. Resolve these before executing the collection repeatedly; otherwise, hard-coded IDs or credentials can make tests fail unpredictably.

Item	What appears in the supplied files	Recommended handling
Token response property names	Postman scripts read responseBody.accessToken and responseBody.refreshToken.	Use these names if that is what the local service returns. Standard OAuth responses often use access_token and refresh_token; do not rename fields blindly.
Refresh token validity	Excel/Steps.txt use 78600 seconds; the Postman collection uses 86400 seconds.	Check the actual client configuration or application code. Do not assume these values are the same.
Role/permission ID note	Steps.txt says get Role ID from permissions and Permission ID from roles; this appears reversed.	roleId must be the ID of a role, and permissionId must be the ID of a permission. Verify response records before calling the assignment endpoint.
Example role name	Steps/Excel use ROLE_CUSTOMER in one example, while the JSON collection creates ROLE_ADMIN.	Use a role that exists in your test environment and matches the test scenario. Avoid duplicate names.
Fixed IDs	Examples use /users/1, /roles/2, /permissions/2.	Prefer IDs extracted from create responses or listed in the live response. Fixed IDs may not exist on a fresh database.
Permission API access	Collection folder notes that testers do not have access to permission/role APIs.	Use an authorized admin/test setup account for setup. Do not expect an ordinary tester token to access administrative endpoints.
Token grant	The collection configures grantTypes=password and submits username/password to /oauth2/token.	Treat this as a legacy/local tutorial flow only. See the security note below before using any OAuth grant in a real product.
Local transport	URLs use http://localhost.	Localhost HTTP is useful for a local sandbox. Use HTTPS and managed secrets outside the local machine.

Important security note about password grant

Your tutorial uses the Resource Owner Password Credentials (password) grant. The IETF OAuth 2.0 Security Best Current Practice, RFC 9700 (January 2025), says this grant MUST NOT be used in new deployments because it exposes the user password to the client and expands the places where credentials can leak. Learn it here only to understand the provided training service; for production user login, use a modern flow such as Authorization Code with PKCE, and for machine-to-machine access evaluate Client Credentials. See References.

3. Endpoint map from your Postman collection

These are the endpoint paths and purposes represented in the uploaded collection and worksheet. The request method, body shape, and authorization requirements must still match the running service's contract.

Folder	Method + endpoint	Purpose
Permission APIs	GET /permissions	List permissions.
Permission APIs	POST /permissions	Create a permission, e.g. read/write.
Permission APIs	GET /permissions/{id}	Read one permission.
Role APIs	GET /roles	List roles.

Folder	Method + endpoint	Purpose
Role APIs	POST /roles	Create a role; source notes duplicate roles may be rejected.
Role APIs	GET /roles/{roleId}	Read one role.
Role APIs	POST /roles/{roleId}/permissions/{permissionId}	Assign a permission to a role.
User APIs	GET /users	List users.
User APIs	POST /users	Create a user.
User APIs	GET /users/{id}	Read one user.
User APIs	POST /users/{userId}/roles/{roleId}	Assign a role to a user.
Client APIs	GET /oauth2/clients	List OAuth clients.
Client APIs	POST /oauth2/clients	Create OAuth client configuration.
Client APIs	GET /oauth2/clients/{id}	Read one OAuth client.
Access Token APIs	POST /oauth2/token	Generate the initial token pair (password grant in this tutorial).
Access Token APIs	POST /oauth2/refresh	Request new tokens using a refresh token.
Employee APIs	GET /employees	Read employee collection; protected.
Employee APIs	POST /employees	Create an employee; protected.
Employee APIs	GET /employees/{id}	Read one employee; protected.
Employee APIs	PUT /employees/{id}	Update an employee; protected.
Employee APIs	DELETE /employees/{id}	Delete an employee; protected.

4. Build the Postman workspace and collection

Step-by-step setup

→ Open Postman and create a workspace named OAuth2 (or use your team workspace).

→ Import OAuth2 Collection.postman_collection.json, or create a collection with the same folders: Permission APIs, Role APIs, User APIs, Client APIs, Access Token APIs, and Employee APIs.

→ Create environment or collection variables so URLs can be changed without editing every request: authBaseUrl = http://localhost:9080 and employeeBaseUrl = http://localhost:9081.

→ Start the application that owns port 9080 (the UserApisWithOauth2/authentication-side application in the supplied steps).

→ Start UserApiOauth2ResourceServer for the Employee APIs on port 9081.

→ Run a simple GET request to confirm each service is reachable. If Postman reports connection refused, confirm the service is running and listening on the expected port before debugging OAuth.

Use variables instead of repeating local URLs

```
{{authBaseUrl}}/oauth2/token
{{authBaseUrl}}/users
{{employeeBaseUrl}}/employees
```

A variable is a named value. It lets the same collection target local, QA, or staging by changing one environment. Use collection or environment variables for values shared by a collection; reserve global variables for values truly needed across collections. Postman supports global, collection, environment, iteration, and local variable scopes.

Helpful Postman test script pattern

```
pm.test("Response is successful", function () {
  pm.expect(pm.response.code).to.be.oneOf([200, 201]);
});

pm.test("Response is JSON when the API returns a JSON body", function () {
  pm.expect(pm.response.headers.get("Content-Type") || "")
    .to.include("application/json");
});
```

Why this matters: a request being sent is not a test passing. A Postman test adds a condition that must be true. Change allowed status codes to the actual contract for that endpoint; do not accept both 200 and 201 without a reason.

5. Permission APIs: create and capture an ID

GET all permissions

```
GET {{authBaseUrl}}/permissions
```

This is a GET request: it reads the permission list and should not create or delete data. Inspect the response body to learn what the service calls each field, especially id, permissionName, and description.

POST create permission

```
POST {{authBaseUrl}}/permissions
Content-Type: application/json

{
  "permissionName": "read write",
  "description": "Admin can access read all APIs"
}
```

Line-by-line explanation: POST asks the service to create a record; Content-Type tells it the request body is JSON; permissionName and description are the two fields in the example payload. The server often returns the newly created record, including its ID, but inspect your response to confirm.

Capture generated ID in Postman

```
const body = pm.response.json();

pm.test("Created permission has an ID", function () {
  pm.expect(body.id).to.exist;
});

if (body.id !== undefined && body.id !== null) {
  pm.collectionVariables.set("permissionId", String(body.id));
}
```

`pm.response.json()` parses the response body into a JavaScript object. `body.id` reads the id field. `pm.collectionVariables.set` stores it so later requests can use `{{permissionId}}`. This avoids hardcoding `/permissions/1`. The source uses `pm.globals.set`; collection variables are usually a narrower and clearer scope for values used by one collection.

```
GET {{authBaseUrl}}/permissions/{{permissionId}}
```

Real-time scenario: when a test creates a permission, the database may assign ID 7 today and ID 12 tomorrow. Capturing the ID keeps the test independent of the database's starting state.

6. Role APIs: create a role and assign a permission

Create a role

```
POST {{authBaseUrl}}/roles
Content-Type: application/json

{
  "roleName": "ROLE_CUSTOMER",
  "description": "New Customer Role"
}
```

The worksheet notes duplicate role names may be rejected. Therefore, a test must decide whether it expects a successful create or a duplicate-name error. Do not run the same create request repeatedly and assume it will always return success.

```
const body = pm.response.json();
if (body.id !== undefined && body.id !== null) {
  pm.collectionVariables.set("roleId", String(body.id));
}
```

Assign permission to role

```
POST {{authBaseUrl}}/roles/{{roleId}}/permissions/{{permissionId}}
```

There is no request body in the source example: both IDs are carried in the URL path. Read the path as “assign this permission to this role.” `roleId` comes from a role record; `permissionId` comes from a permission record. The source note that reverses these IDs appears to be a typo, so confirm each actual record before sending this request.

> **Best practice**
>
> Capture IDs from API responses and name variables to reflect their meaning. An integer value of 2 is not enough information; `roleId=2` and `permissionId=2` are different concepts even if their numeric values happen to match.

7. User APIs: create a user and assign a role

Create user

```
POST {{authBaseUrl}}/users
Content-Type: application/json

{
  "username": "sai_api_test_001",
  "password": "Use-a-local-test-secret",
  "email": "sai_api_test_001@example.test",
  "firstName": "Saikiran",
  "lastName": "Biradar",
  "roles": ["ROLE_ADMIN"]
}
```

The payload includes a username, password, email, name fields, and a roles array. The example values above are placeholders for an isolated local environment; use the exact schema supported by your application. Never reuse a real personal password in test data. Use a unique test username/email for every create test if duplicates are rejected.

```
const body = pm.response.json();
if (body.id !== undefined && body.id !== null) {
  pm.collectionVariables.set("userId", String(body.id));
}
```

```
GET {{authBaseUrl}}/users/{{userId}}
```

```
POST {{authBaseUrl}}/users/{{userId}}/roles/{{roleId}}
```

The first request retrieves the user. The second associates the user with a role. If your create-user body already contains roles, confirm whether the separate assignment is needed; some systems may reject duplicate assignments or treat them as idempotent.

Real-time scenarios for user APIs

Scenario	Test idea	Expected behavior to confirm
Unique user registration	Create user with an unused username and email.	API returns documented success and a usable ID.
Duplicate username	Repeat create with the same username.	Documented validation/conflict error; no second account created.
Missing username	Omit username from JSON.	Validation failure such as 400, based on API contract.
Role assignment	Create/locate user and role, then assign using IDs.	Success response and the role is visible in user details, if returned.
Unauthorized admin action	Call an admin-only endpoint with a tester token.	Access is denied; confirm whether this service uses 403 or another documented response.

8. OAuth client APIs

Create a local demo client

```
POST {{authBaseUrl}}/oauth2/clients
Content-Type: application/json

{
  "clientId": "employee_client",
  "clientSecret": "LOCAL_ONLY_DEMO_SECRET",
  "clientName": "employee_client",
  "grantTypes": "password",
  "scopes": "read write",
  "accessTokenValidity": 600,
  "refreshTokenValidity": 78600
}
```

The source describes accessTokenValidity=600 seconds (10 minutes). The refreshTokenValidity differs across files: 78600 in the spreadsheet/steps and 86400 in the Postman JSON. Use the value configured in the running application, not whichever one you last saw in an example.

Client secrets identify a client to the authorization server. In a real system they must be protected like passwords. Do not commit them to Git, embed them in a public mobile app, or include them in shared Postman exports. Local demo secrets are not production secrets.

9. Generate the access token

Postman request settings

```
POST {{authBaseUrl}}/oauth2/token
Body type: x-www-form-urlencoded

 grant_type    = password
 client_id     = employee_client
 client_secret = {{clientSecret}}
 username      = {{testUsername}}
 password      = {{testPassword}}
 scope         = read write
```

The form must be URL-encoded, not a raw JSON object, because that is what your supplied collection and Steps.txt specify. Postman converts each key/value pair into the form request body. The endpoint shown by your source is a custom local route at `/oauth2/token`.

Capture access and refresh tokens

```
const body = pm.response.json();

pm.test("Token response contains an access token", function () {
  pm.expect(body.accessToken).to.be.a("string").and.not.empty;
});

pm.test("Token response contains a refresh token", function () {
  pm.expect(body.refreshToken).to.be.a("string").and.not.empty;
});

if (body.accessToken) {
  pm.collectionVariables.set("accessToken", body.accessToken);
}
if (body.refreshToken) {
  pm.collectionVariables.set("refreshToken", body.refreshToken);
}
```

The scripts in your Postman collection store the same values globally. This example uses collection variables so the token pair belongs to this collection. Do not print the values to the Postman Console or attach them to bug reports.

What to check after the token request

- HTTP status matches the local token endpoint contract (often 200 for a token request, but verify the implementation).
- Response body is JSON and contains a non-empty accessToken value.
- Refresh token is present if the server issues one.
- Access token works against the Employee API.
- Invalid user password, invalid client secret, unsupported grant type, and invalid scope are rejected as documented.

Naming difference to remember

The local tutorial reads `accessToken` and `refreshToken` in camelCase. Many OAuth servers use `access_token` and `refresh_token` in snake_case. Parse exactly what your server returns; do not copy a response parser from another server without checking the JSON.

10. Refresh an expired access token

The access token in the client example is configured for a short lifetime. When it expires, requests to the resource server should stop succeeding until the client obtains a new access token. Your local service exposes `/oauth2/refresh` and expects a JSON body.

```
POST {{authBaseUrl}}/oauth2/refresh
Content-Type: application/json

{
  "refreshToken": "{{refreshToken}}"
}
```

```
const body = pm.response.json();

pm.test("Refresh response contains an access token", function () {
  pm.expect(body.accessToken).to.be.a("string").and.not.empty;
});

if (body.accessToken) {
  pm.collectionVariables.set("accessToken", body.accessToken);
}
```

```
if (body.refreshToken) {
  // Store a new refresh token if this response rotates it.
  pm.collectionVariables.set("refreshToken", body.refreshToken);
}
```

If the server rotates the refresh token, replace the stored value with the newly returned one. Reusing an old refresh token might fail. If the refresh endpoint returns no new refreshToken, do not overwrite the saved value with an empty or undefined value.

Token lifecycle scenario

→ Generate the access/refresh token pair.
→ Call GET /employees and confirm the protected request works.
→ Wait until the access token expires, or configure a short lifetime in the local test system.
→ Call GET /employees with the expired access token; confirm the service rejects it according to its contract.
→ Call POST /oauth2/refresh with the stored refresh token.
→ Replace stored tokens and retry GET /employees.
→ Test an invalid or expired refresh token; confirm the endpoint rejects it and does not issue a valid access token.

11. Employee APIs: the protected resource server

Your steps explicitly say to start the `UserApiOauth2ResourceServer` application before testing Employee APIs. The resource server is at `http://localhost:9081`; the token is obtained from the service at port 9080.

First call without a token

```
GET http://localhost:9081/employees

Expected in the supplied tutorial: 401 Unauthorized
```

This is a valuable negative test: it proves the resource is protected. If it unexpectedly returns data without credentials, investigate whether authentication is disabled or the endpoint is intentionally public.

Call with Bearer token

```
GET {{employeeBaseUrl}}/employees
Authorization: Bearer {{accessToken}}
```

The header transports the access token. In Postman, choose Authorization → OAuth 2.0, set the token value to `{{accessToken}}`, and ensure the token is added to the request header. Alternatively, set the Authorization header manually. Do not do both if your client would generate duplicate Authorization headers.

Create employee

```
POST {{employeeBaseUrl}}/employees
Authorization: Bearer {{accessToken}}
Content-Type: application/json

{
  "name": "Sneha",
  "email": "sneha_test@example.test",
  "department": "QA Engineer",
  "salary": 139999,
  "createdAt": "2026-10-14T08:34:57.637Z"
}
```

The date-time is an ISO-8601 example from the source material. If the field is generated by the application, do not send it unless the API requires it. Create endpoints often respond with the new record and an ID; capture that ID to query, update, and delete the exact resource.

Read, update, and delete

```
GET    {{employeeBaseUrl}}/employees/{{employeeId}}
PUT    {{employeeBaseUrl}}/employees/{{employeeId}}
DELETE {{employeeBaseUrl}}/employees/{{employeeId}}
```

For PUT, the source supplies all employee fields while changing the name. That suggests the endpoint may expect a full representation. Do not send only the name unless the API contract says partial updates are supported (commonly PATCH).

Suggested tests for each Employee endpoint

Operation	Positive test	Negative/security test
GET all employees	Valid token returns the documented collection response.	Missing, malformed, invalid, or expired token is rejected.
POST employee	Valid token + valid body creates one record and returns its ID.	Missing required field, invalid email, negative salary, or no token is rejected.
GET employee by ID	Created ID returns the matching employee.	Non-existent ID gives documented not-found response.
PUT employee	Update a field and read again to verify persistence.	Invalid body or an ID that does not exist is rejected as documented.
DELETE employee	Delete a disposable test record and verify it is absent.	No token or insufficient permission must not delete the record.

12. Rest Assured setup (Java + Maven + TestNG)

Example folder structure

```
UserApisWithOauth2-tests/
  pom.xml
  src/test/java/
    support/OAuthTokenSupport.java
    tests/EmployeeApiTest.java
  src/test/resources/
    test-data/   (optional JSON payload files)
```

Keep token creation logic in a support/helper class rather than duplicating the same token request in every test. Put test methods in test classes; keep environment-specific URLs out of business assertions.

Maven dependencies (example)

```
<dependencies>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>6.0.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.11.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

This dependency snippet is an example for a Java 17+ project. The versions are examples, not a claim that they are the newest. Use versions approved for your team and runtime; if your POM already works, add only missing dependencies and avoid duplicate versions.

Environment configuration

Configure the following as environment variables in your IDE run configuration or CI secret store. Do not put real secrets in Java source code, a checked-in JSON file, or a shared collection export.

Environment variable	Meaning	Local-demo example value
AUTH_BASE_URL	Authorization service base URL	http://localhost:9080
EMPLOYEE_BASE_URL	Employee resource server URL	http://localhost:9081
OAUTH_CLIENT_ID	OAuth client id	employee_client
OAUTH_CLIENT_SECRET	OAuth client secret	Set privately for local sandbox
OAUTH_USERNAME	Test user	Sai (only if that user exists locally)
OAUTH_PASSWORD	Test user password	Set privately for local sandbox

Run and troubleshoot your test project

- Start both local applications: the authorization service on port 9080 and `UserApiOauth2ResourceServer` on port 9081.
- Check your Java and Maven installations with `java -version` and `mvn -version`. The examples assume a compatible Java version and an existing Maven project.

```
# From the Maven project root
mvn test

# Run one TestNG class only
mvn -Dtest=EmployeeApiTest test
```

- If the test fails before reaching an assertion, verify the base URLs, ports, environment-variable names, credentials, client configuration, and whether the user has the required role/scope.
- Never paste real secrets into shell commands that may be saved in terminal history or into a shared log. Use your IDE run configuration or a CI secret store.

13. Code explained: reusable token support

This class requests a token once and returns the access-token string. It checks that the expected response field exists and fails with a useful error if authentication did not work. It intentionally avoids printing the token.

```
package support;

import io.restassured.http.ContentType;
import io.restassured.response.Response;

import static io.restassured.RestAssured.given;

public final class OAuthTokenSupport {
   private OAuthTokenSupport() { }

   private static String requiredEnv(String name) {
      String value = System.getenv(name);
      if (value == null || value.isBlank()) {
         throw new IllegalStateException(
            "Missing required environment variable: " + name);
      }
      return value;
   }

   public static String requestAccessToken() {
      String authBaseUrl = System.getenv().getOrDefault(
         "AUTH_BASE_URL", "http://localhost:9080");

      Response response = given()
         .baseUri(authBaseUrl)
         .contentType(ContentType.URLENC)
         .formParam("grant_type", "password")
         .formParam("client_id", requiredEnv("OAUTH_CLIENT_ID"))
         .formParam("client_secret", requiredEnv("OAUTH_CLIENT_SECRET"))
         .formParam("username", requiredEnv("OAUTH_USERNAME"))
         .formParam("password", requiredEnv("OAUTH_PASSWORD"))
         .formParam("scope", "read write")
      .when()
         .post("/oauth2/token");

      if (response.statusCode() != 200) {
         throw new IllegalStateException(
            "Token request failed with HTTP " + response.statusCode());
      }

      String token = response.jsonPath().getString("accessToken");
      if (token == null || token.isBlank()) {
         throw new IllegalStateException(
            "Token response did not contain a non-empty accessToken field");
      }
      return token;
   }
}
```

Code explanation

Code part	What it means
`requiredEnv("OAUTH_CLIENT_ID")`	Reads the configured variable. If it is missing, the test stops immediately with a clear message.
`ContentType.URLENC`	Tells Rest Assured to encode form fields like the Postman x-www-form-urlencoded body.
`.formParam(key, value)`	Adds one form field to the token request.
`.post("/oauth2/token")`	Sends the HTTP POST to the auth base URL plus the token path.
`response.statusCode()`	Reads the HTTP status. This example expects 200; change it if your local token endpoint specifies another status.

Code part	What it means
`response.jsonPath().getString("accessToken")`	Reads the accessToken JSON field from the response.
`return token`	Gives the token back to the test so it can authorize Employee API calls.

Important limitation

This method intentionally reproduces the password-grant flow in the supplied local tutorial. It is for learning/testing that specific service, not a recommendation for a new production OAuth client.

14. Code explained: call a protected Employee API

The first test deliberately omits the token. The second obtains a token from the helper in Section 13, attaches it as Bearer authentication, and checks a successful JSON response. The test methods are independent: the negative test does not need the token endpoint to be available.

```
import org.testng.annotations.Test;
import static io.restassured.RestAssured.given;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.containsString;
import static org.hamcrest.Matchers.isEmptyOrNullString;
import static org.hamcrest.Matchers.not;

public class EmployeeApiTest {
    private String employeeBaseUrl() {
        return System.getenv().getOrDefault(
            "EMPLOYEE_BASE_URL", "http://localhost:9081");
    }

    @Test
    public void rejectsMissingToken() {
        given().baseUri(employeeBaseUrl())
            .when().get("/employees")
            .then().statusCode(401);
    }

    @Test
    public void validTokenCanReadEmployees() {
        String token = support.OAuthTokenSupport.requestAccessToken();
        String body = given().baseUri(employeeBaseUrl())
            .auth().oauth2(token)
            .when().get("/employees")
            .then().statusCode(200)
            .header("Content-Type", containsString("application/json"))
            .extract().asString();
        assertThat(body, not(isEmptyOrNullString()));
    }
}
```

Why this code works

- `rejectsMissingToken` does not set an Authorization header; it checks the 401 behavior described in the supplied steps.
- `requestAccessToken()` obtains the token using the local token flow from Section 13. In a production system, use a supported, secure grant for the client type.
- `.auth().oauth2(token)` makes Rest Assured send Bearer authentication. Do not prepend `Bearer ` yourself when using this method.
- The body assertion confirms the response is not empty. Add field-level assertions after you inspect the actual employee JSON schema.

15. Full Employee CRUD example: end-to-end scenario

CRUD means Create, Read, Update, Delete. A useful integration test creates its own employee, captures the generated ID, verifies that the record can be read, updates it, verifies the update, and cleans up. This example shows the sequence; align success status codes with the contract documented by the service.

```
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.Test;
import java.time.OffsetDateTime;
import java.util.HashMap;
import java.util.Map;
import java.util.UUID;

import static io.restassured.RestAssured.given;
import static org.hamcrest.Matchers.equalTo;

public class EmployeeCrudFlowTest {
    private String token;
    private final String baseUrl = System.getenv().getOrDefault(
        "EMPLOYEE_BASE_URL", "http://localhost:9081");

    @BeforeClass
    public void setup() {
        token = support.OAuthTokenSupport.requestAccessToken();
    }

    @Test
    public void createReadUpdateDeleteEmployee() {
        String uniqueEmail = "api-" + UUID.randomUUID() + "@example.test";
        Integer employeeId = null;

        Map<String, Object> createBody = new HashMap<>();
        createBody.put("name", "API Test Employee");
        createBody.put("email", uniqueEmail);
        createBody.put("department", "Quality Engineering");
        createBody.put("salary", 90000);
        createBody.put("createdAt", OffsetDateTime.now().toString());

        Response created = given()
            .baseUri(baseUrl).auth().oauth2(token)
            .contentType(ContentType.JSON).body(createBody)
        .when()
            .post("/employees");

        // Capture the ID before asserting the status so cleanup can still be attempted.
        try {
            if (created.jsonPath().get("id") != null) {
                employeeId = created.jsonPath().getInt("id");
            }
            // Confirm the actual API contract: some APIs return 201, others 200.
            created.then().statusCode(201);
            if (employeeId == null) {
                throw new IllegalStateException("Create response did not contain an employee id");
            }

            given().baseUri(baseUrl).auth().oauth2(token)
                .pathParam("id", employeeId)
            .when().get("/employees/{id}")
            .then().statusCode(200).body("id", equalTo(employeeId));

            createBody.put("name", "API Test Employee Updated");
            given().baseUri(baseUrl).auth().oauth2(token)
                .contentType(ContentType.JSON).body(createBody)
                .pathParam("id", employeeId)
            .when().put("/employees/{id}")
            .then().statusCode(200); // Adjust to the API contract.

            given().baseUri(baseUrl).auth().oauth2(token)
                .pathParam("id", employeeId)
            .when().get("/employees/{id}")
            .then().statusCode(200)
                .body("name", equalTo("API Test Employee Updated"));

            given().baseUri(baseUrl).auth().oauth2(token)
```

```
            .pathParam("id", employeeId)
        .when().delete("/employees/{id}")
        .then().statusCode(204); // Adjust to the API contract.
        employeeId = null; // Already deleted and verified; do not delete twice.
      } finally {
        // If an earlier assertion fails, attempt cleanup to avoid test pollution.
        if (employeeId != null) {
          given().baseUri(baseUrl).auth().oauth2(token)
              .pathParam("id", employeeId)
          .when().delete("/employees/{id}");
        }
      }
    }
}
```

Do not blindly copy status codes

Your Postman collection shows methods and payloads but does not include successful response examples for every Employee operation. The code uses common example statuses (201 for create, 200 for read/update, 204 for delete). Verify these against the actual API contract; change them when your service documents a different result.

Walk through the CRUD test

Step	What the code does	Why it matters
1. Unique email	Creates an email using UUID.randomUUID().	Avoids collisions when the test is rerun.
2. Request body	Uses a Map to build JSON fields.	Keeps the example readable and allows field updates.
3. POST	Sends createBody with a Bearer token.	Creates a record and expects the documented success status.
4. Capture ID	Reads the generated id from the response.	All later calls use the actual new resource, not a hard-coded ID.
5. GET	Reads `/employees/{id}` and checks returned id.	Confirms the newly created record is retrievable.
6. PUT	Changes name and sends the update.	Tests that modification is accepted.
7. Verify update	Reads the record again and checks the changed name.	Tests persistence, not just an HTTP success code.
8. DELETE + cleanup	Deletes the record and attempts cleanup in finally.	Prevents disposable test data accumulating if the test fails.

16. Refresh-token automation in Rest Assured

```
import io.restassured.http.ContentType;
import io.restassured.response.Response;
import java.util.Map;
import static io.restassured.RestAssured.given;

public class OAuthRefreshSupport {
    public static TokenPair refresh(String refreshToken) {
        String baseUrl = System.getenv().getOrDefault(
            "AUTH_BASE_URL", "http://localhost:9080");

        Response response = given()
            .baseUri(baseUrl)
            .contentType(ContentType.JSON)
            .body(Map.of("refreshToken", refreshToken))
        .when()
            .post("/oauth2/refresh");

        if (response.statusCode() != 200) {
```

```
            throw new IllegalStateException(
                "Refresh failed with HTTP " + response.statusCode());
        }

        String newAccessToken = response.jsonPath().getString("accessToken");
        String newRefreshToken = response.jsonPath().getString("refreshToken");
        if (newAccessToken == null || newAccessToken.isBlank()) {
            throw new IllegalStateException("Refresh response has no accessToken");
        }
        return new TokenPair(newAccessToken, newRefreshToken);
    }

    public record TokenPair(String accessToken, String refreshToken) { }
}
```

This code mirrors the custom refresh endpoint in the uploaded collection. `Map.of` creates a small JSON object with a single `refreshToken` field. The method returns both values because a server may rotate the refresh token. A Java record is a concise immutable carrier for those two strings; if your Java version is older than 16, replace it with a normal class.

The example uses HTTP 200 as the expected refresh success code. Check the local endpoint contract; if it returns a different documented code, adjust that assertion. Also ensure the actual token endpoint field names match the response parser.

17. Negative testing and real-world failure scenarios

Scenario	Likely cause	How to test / investigate
401 on Employee API	No Authorization header, invalid token, expired token, wrong token type.	Send no token first, then valid token. Check `Authorization: Bearer ...`; never print the token.
401 even with a token	Token generated by a different service/environment, malformed header, token expired.	Confirm auth URL is port 9080 and resource URL is port 9081; refresh token; check whether the server expects Bearer auth.
403 with a valid token	Caller is authenticated but lacks permission/scope or role.	Try the endpoint with an authorized role and compare scopes; confirm expected access-control rules.
400 on token request	Wrong form encoding, missing parameters, invalid grant_type, or invalid credentials.	Use x-www-form-urlencoded exactly as the tutorial specifies; inspect non-secret error detail.
Token response has no `accessToken`	Server returns different property names or error JSON.	Inspect response structure; use `access_token` only if that is what your service returns.
Refresh request fails	Refresh token expired/revoked/rotated or wrong JSON key.	Use most recently issued refresh token and check the custom `/oauth2/refresh` contract.
Connection refused	Application not started or wrong port.	Start auth service on 9080 and resource server on 9081; test connectivity before auth debugging.
Duplicate username or role	Test data already exists from previous run.	Create unique test data or explicitly test expected duplicate rejection.
Wrong resource is read/updated	Hard-coded ID 1 points to another user's fixture.	Capture IDs from create responses; isolate data and clean up.
Test passes despite wrong business data	Only status code is asserted.	Also validate response ID, fields, persistence after update, and record absence after deletion.

18. Best practices for an industry-style framework

Area	Best practice	Reason
Secrets	Read client secret/user password from environment or a secret manager.	Prevents credentials being committed or shared accidentally.
Token logging	Never print access/refresh tokens or client secrets. Use logs only for safe metadata.	Bearer tokens function like credentials while valid.
Variable scope	Use environment variables for URLs/secrets and collection variables for flow IDs/tokens.	Reduces accidental interference between unrelated collections.
HTTPS	Use HTTPS outside local development.	Protects credentials and tokens in transit.
Token design	For production user login, do not use password grant. Choose a flow that suits the client, such as Authorization Code + PKCE.	Aligns with modern OAuth security guidance.
Assertions	Assert status code plus meaningful body/headers and persistence.	HTTP 200 alone does not prove business correctness.
IDs	Extract IDs from responses, do not hard-code 1 or 2.	Test data order and database state are not guaranteed.
Data setup	Create unique test records and clean them up.	Allows safe reruns and avoids test pollution.
Test independence	Avoid relying on TestNG priority as the only dependency mechanism. Keep required setup explicit.	Order-based tests can break when run separately or in parallel.
Timeouts	Use a documented performance budget and realistic upper bound.	Avoids arbitrary timing assertions that fail on noisy CI machines.
Contract alignment	Use the API spec to determine status codes, body schema, and error format.	Different APIs may return different valid outcomes.
Least privilege	Give a test account only the roles and scopes needed for the scenario.	Tests access control more safely and catches over-privileged tokens.
Token scope	Use read-only tokens for read tests where supported; use write scope only for mutation tests.	Keeps test credentials limited to the action being verified.
CI setup	Inject secrets via the CI secret store and pass URLs as configuration.	The same suite can run in local, QA, and staging environments.

Avoid these common mistakes

- Using the access token in a URL query parameter instead of the Authorization header.
- Adding the word `Bearer ` manually when also using Rest Assured `.auth().oauth2(token)`.
- Using a token from one environment against a resource server in another environment.
- Sharing the exported Postman collection with real secrets or live token values.
- Hard-coding the IDs of users, roles, permissions, or employees.
- Assuming the token is a JWT because a variable is named accessToken; the source does not show its actual format.
- Assuming the response is correct just because the status code is 200.
- Using a production-like admin account for every automated test instead of testing least-privilege roles.

19. Hands-on exercises

Try each task before reading the answer key. Use only your local demo services, with permission to create and delete test data.

Beginner 1: Create a Postman GET request to list permissions. Record the response shape and the `id` field.

Beginner 2: Create one permission with POST, validate that its ID exists, and save it to `permissionId`.

Beginner 3: Create a role and save its returned ID to `roleId`. Avoid using an existing duplicate role name.

Beginner 4: Call GET `/employees` without an Authorization header and confirm the protected endpoint rejects it as described in the source steps.

Beginner 5: Generate a token using the supplied form fields, then attach it to GET `/employees`.

Intermediate 6: Call `/oauth2/refresh` with the saved refresh token. Replace the current access token with the newly returned token.

Intermediate 7: Write a Rest Assured test that checks a protected employee API fails without a token and succeeds with a token.

Intermediate 8: Create an employee, capture its ID, GET it, update its name, and GET it again to verify the new name.

Intermediate 9: Repeat create-user with a duplicate username and document the actual error contract.

Advanced 10: Use a role that has read-only access. Verify GET works while a write operation is denied if the application implements those permissions.

Advanced 11: Test a missing/invalid/expired access token and an expired refresh token. Record response code and error body without leaking secrets.

Advanced 12: Refactor repeated URL, token, and common response checks into reusable Rest Assured helper methods. Keep business assertions in test methods.

20. Practice solutions / answer patterns

Solution: extract and save a created ID in Postman

```
const body = pm.response.json();
pm.test("ID was returned", () => pm.expect(body.id).to.exist);
if (body.id !== undefined && body.id !== null) {
  pm.collectionVariables.set("permissionId", String(body.id));
}
```

Solution: authenticated GET with Rest Assured

```
given()
    .baseUri(System.getenv().getOrDefault(
        "EMPLOYEE_BASE_URL", "http://localhost:9081"))
    .auth().oauth2(accessToken)
.when()
    .get("/employees")
.then()
    .statusCode(200);
```

Read this as: configure the base URL, attach the token, send GET `/employees`, and assert the expected status. Add body checks once you know the response JSON shape.

Solution: dynamic path parameter

```
given()
    .baseUri("http://localhost:9080")
    .pathParam("roleId", roleId)
    .pathParam("permissionId", permissionId)
.when()
    .post("/roles/{roleId}/permissions/{permissionId}")
.then()
    .statusCode(200); // Use the actual success code from the API contract.
```

Path parameters make the route readable and remove manual string concatenation. The status code shown is only a placeholder pattern until you verify the assignment endpoint's actual success status.

Solution: check for updated employee field

```
given()
   .baseUri(employeeBaseUrl)
   .auth().oauth2(accessToken)
   .pathParam("id", employeeId)
.when()
   .get("/employees/{id}")
.then()
   .statusCode(200)
   .body("name", equalTo("Updated Name"));
```

This tests the result after an update, not merely the update endpoint's response. That difference is important in integration testing.

21. Quick revision checklist

- ☐ I can explain why the project has an auth service on port 9080 and an employee resource server on port 9081.
- ☐ I can generate accessToken and refreshToken using the local tutorial request shape.
- ☐ I know the provided flow uses x-www-form-urlencoded for `/oauth2/token` and JSON for `/oauth2/refresh`.
- ☐ I can store token and ID values in collection variables without hard-coding IDs.
- ☐ I can explain why the Employee API returns 401 without a token in the supplied steps.
- ☐ I can send a Bearer token from Postman and Rest Assured.
- ☐ I can test missing, invalid, expired, and insufficient-permission scenarios.
- ☐ I can create, read, update, and delete a test employee while cleaning up test data.
- ☐ I do not log secrets or tokens and do not commit them to Git.
- ☐ I understand the password-grant flow is present in the tutorial but is not recommended for production use.

22. Glossary

Term	Meaning
Access token	Credential sent to the resource server to access protected resources.
Authorization server	Service responsible for issuing tokens after validating a client/user according to its policy.
Bearer token	A token presented by whoever possesses it; protect it as a credential.
Client	Application or test client requesting a token or calling an API.
Client secret	Confidential credential associated with a client.
Grant type	The method by which a client requests authorization/token issuance.
Refresh token	Credential used to obtain a new access token, subject to server policy.
Resource server	API server that accepts tokens and serves protected data.
Scope	Named permission set requested or granted to a token.
Test data isolation	Creating, naming, and cleaning up test records so tests do not collide.
401 Unauthorized	Usually missing or rejected authentication.
403 Forbidden	Usually authenticated caller is not allowed to perform the action.

23. References and source notes

Primary sources supplied in this conversation:

- OAuth2 Collection.postman_collection.json — request folders and examples for Permission, Role, User, Client, Access Token, and Employee APIs.
- OAuth2 Flow To Generate Access Token.xlsx — endpoint inventory, example payloads, notes, and token form fields.
- Steps.txt — suggested Postman setup and the order for generating tokens and calling protected Employee APIs.

External best-practice references used for additional security context:

- IETF RFC 9700, Best Current Practice for OAuth 2.0 Security (January 2025): https://www.rfc-editor.org/rfc/rfc9700.html
- Postman documentation, reference variables in scripts: https://learning.postman.com/docs/tests-and-scripts/write-scripts/postman-sandbox-reference/pm-variables/

Source-based statements in this workbook describe the supplied local training project. Recommendations about safer OAuth deployment and variable scope are separately marked as best practices. Because the services are local, verify all successful status codes and exact response schemas by running the application and inspecting the actual responses.